

CRASH COURSE PLAN

React Crash Course

| Lộ trình học chi tiết từng phần — kèm giải thích cơ chế render bên trong

Repo: [crash-course / react-crash-course-traversy](#)

Tài liệu bám theo **React 19.2** (bản ổn định mới nhất) + **Vite** + **React Router v7/v8** — kiểm chứng theo [react.dev](#) & [reactrouter.com](#)

Dự án hiện tại của bạn: `react 19.2.5 · react-router-dom 7.14.2 · vite 8 · tailwindcss 4`

Mục lục

JS	JavaScript ES6+ cho React	NỀN TẢNG NGÔN NGỮ
00	React hoạt động thế nào (render & reconciliation)	NỀN TẢNG
01	Khởi tạo dự án với Vite & cấu trúc thư mục	
02	JSX, Component & Props	
03	State với useState — tư duy bất biến	
04	Render danh sách, key & render có điều kiện	
05	Sự kiện & Form (controlled component)	
06	useEffect — dùng đúng và dùng ít	
07	Gọi API & Custom Hook	
08	Chia sẻ state: lifting up, Context, useReducer	
09	React Router (routing, loader, form action)	
10	Hiệu năng: re-render, memo & React Compiler	
11	React 19: Actions, use(), form hooks	
12	Kết nối API thật (Laravel / NestJS) & build production	
13	Testing với Vitest + Testing Library	TUỶ CHỌN

Cách dùng tài liệu này: Bạn tự code 100% từng phần. Mỗi phần có mục "Việc cần làm" (task) — không có code giải sẵn để copy, chỉ có ví dụ cú pháp minh họa để hiểu cách viết. Khi xong một phần, quay lại chat để mình review và giải thích tiếp phần sau.

Dự án xuyên suốt: App quản lý tin tuyển dụng (job listings) — danh sách, chi tiết, thêm, sửa, xoá. Đúng dự án bạn đã làm theo Traversy, nhưng lần này đi sâu vào **vì sao** mỗi thứ hoạt động như vậy. Ở Part 12 ta sẽ thay json-server bằng **API Laravel/NestJS thật** mà bạn tự viết — đó là lúc bạn thành fullstack thật sự.

JavaScript ES6+ cho React

Mục tiêu: Nắm chắc các tính năng JS mà React dùng ở **mọi dòng code**. Phần lớn lỗi "React khó hiểu" thực chất là lỗi chưa vững JS — đặc biệt là **tham chiếu vs giá trị**, **closure** và **bất biến (immutability)**.

J.1 let / const và scope

```
const user = { name: 'Nhat' }; // const = KHÔNG gán lại được biến
user.name = 'Long';           // ✓ vẫn sửa được thuộc tính bên trong!
// user = {}                  ✗ TypeError: Assignment to constant variable

let count = 0; count++;        // let khi cần gán lại
// var: KHÔNG dùng nữa (scope theo function, bị hoisting khó lường)
```

Quy tắc: mặc định dùng const, chỉ đổi sang let khi thật sự cần gán lại. Trong React bạn gần như chỉ dùng const.

J.2 Tham chiếu vs giá trị — gốc rễ của mọi bug React

```
// Primitive (number, string, boolean, null, undefined) → so sánh theo GIÁ TRỊ
1 === 1           // true
'a' === 'a'       // true

// Object / Array / Function → so sánh theo THAM CHIẾU (địa chỉ ô nhớ)
[] === []         // false !!
{} === {}         // false !!
const a = [1]; const b = a; a === b // true (cùng 1 địa chỉ)
```

Vì sao cực kỳ quan trọng: React quyết định "có cần render lại không" bằng phép so sánh `Object.is` (gần như `===`). Nên: (1) sửa trực tiếp mảng/object trong state → React thấy **vẫn cùng tham chiếu** → **không render lại**; (2) truyền `{}` hay `[]` mới mỗi lần render vào props/dependency → React thấy **luôn khác** → render lại / chạy effect vô hạn. Hai lỗi này chiếm phần lớn bug của người mới.

J.3 Destructuring, spread & rest

```
// Destructuring object – cách nhận props chuẩn của React
const { title, salary = 'Thoả thuận' } = job; // có giá trị mặc định
const { company: { name } } = job; // lồng nhau

// Destructuring array – cách useState trả về
const [count, setCount] = useState(0);

// Spread – TẠO BẢN SAO MỚI (cốt lõi của immutability)
const updated = { ...job, title: 'Mới' }; // copy object + ghi đè 1 field
const appended = [...jobs, newJob]; // copy mảng + thêm phần tử

// Rest – gom phần còn lại
const { id, ...rest } = job;
function Button({ children, ...props }) { ... } // pattern rất hay dùng
```

Spread chỉ copy NÔNG (shallow): `{...job}` copy được field cấp 1, nhưng `job.company` vẫn là **cùng một object**. Muốn sửa dữ liệu lồng nhau phải spread từng cấp: `{ ...job, company: { ...job.company, name: 'X' } }`.

J.4 Arrow function & this

```
const double = (x) => x * 2; // return ngầm
const make = (x) => ({ value: x }); // trả object → PHẢI bọc ()
const sum = (a, b) => { return a + b; }; // có {} thì phải return tường minh
```

Arrow function **không có this riêng** — nó mượn this của scope bao ngoài. Với React function component (kiểu hiện đại) bạn gần như không đụng tới this nữa; chỉ cần nhớ điều này khi đọc code class component cũ.

J.5 Các method mảng phải thuộc lòng

Method	Trả về	Dùng trong React để
<code>map()</code>	Mảng mới cùng độ dài	Render danh sách — dùng nhiều nhất
<code>filter()</code>	Mảng mới ít phần tử hơn	Xoá item khỏi state, lọc kết quả
<code>find()</code>	1 phần tử hoặc <code>undefined</code>	Tìm item theo id
<code>some()</code> / <code>every()</code>	boolean	Kiểm tra điều kiện tồn tại / toàn bộ
<code>reduce()</code>	Giá trị gộp bất kỳ	Tính tổng, gom nhóm
<code>slice()</code>	Mảng mới	An toàn — copy một đoạn
<code>splice()</code> , <code>push</code> , <code>sort</code> , <code>reverse</code>	Sửa tại chỗ	KHÔNG dùng trực tiếp trên state — phải copy trước

```
// ✗ SAI – sửa tại chỗ, React không thấy thay đổi
jobs.push(newJob); setJobs(jobs);
jobs.sort((a, b) => a.title.localeCompare(b.title));

// ✓ ĐÚNG – luôn tạo mảng mới
setJobs([...jobs, newJob]);
setJobs([...jobs].sort((a, b) => a.title.localeCompare(b.title)));
setJobs(jobs.filter((j) => j.id !== id));
```

J.6 Toán tử hay dùng trong JSX

```
{ isLoading && <Spinner /> } // short-circuit: true thì render
{ jobs.length > 0 ? <List /> : <Empty /> } // ternary: if/else
{ job?.company?.name } // optional chaining: null thì undefined
{ job.salary ?? 'Thỏa thuận' } // nullish: chỉ thay khi null/undefined
```

Bẫy kinh điển: `{jobs.length && <List />}` — nếu `length === 0`, JS trả về **số 0** và React sẽ **in ra chữ "0"** trên màn hình. Luôn ép sang boolean: `{jobs.length > 0 && ...}`. Ngoài ra: `??` khác `||` — `||` thay cả `0` và `' '` (falsy), `??` chỉ thay null/undefined.

J.7 Module: import / export

```
// export mặc định – 1 cái/file, import đặt tên tùy ý
export default function JobCard() { }
import JobCard from './components/JobCard';

// named export – nhiều cái/file, import phải đúng tên
export const API_URL = '/api/jobs';
export function formatSalary() { }
import { API_URL, formatSalary as fmt } from './utils';
```

J.8 Bất đồng bộ: Promise, async/await, fetch

```
async function getJobs() {
  try {
    const res = await fetch('/api/jobs');

    // QUAN TRỌNG: fetch KHÔNG throw khi server trả 404/500 – phải tự kiểm tra
    if (!res.ok) throw new Error(`HTTP ${res.status}`);

    return await res.json();
  } catch (err) {
    console.error(err);
    throw err;
  }
}
```

Ba trạng thái của Promise: **pending** → **fulfilled / rejected**. `await` chỉ dùng được trong hàm `async`, và hàm `async` **luôn trả về Promise**. Đây là lý do bạn **không được** khai `async` cho callback của `useEffect` (Part 06).

J.9 Closure — nền tảng để hiểu Hooks

```
function counter() {  
  let count = 0;           // biến này "sống" nhờ closure  
  return () => ++count;     // hàm trong nhớ được biến ngoài  
}  
const inc = counter();  
inc(); inc();              // 1, 2
```

Closure = hàm "nhớ" được môi trường nơi nó được tạo ra. React dùng closure cho **toàn bộ** hệ thống hook: mỗi lần render tạo ra một closure mới với giá trị state của lần render đó. Hiểu điều này là hiểu vì sao có hiện tượng **"stale closure"** — hàm trong `setTimeout` hay `useEffect` nhìn thấy giá trị state **cũ** của lần render mà nó được tạo (Part 03 & 06).

J.10 Bảng tra: JS ↔ PHP (cho người học fullstack)

JavaScript	PHP
<code>const x = 1</code>	<code>\$x = 1;</code>
<code>obj.prop</code>	<code>\$obj->prop</code>
<code>{ a: 1 }</code> (object)	<code>['a' => 1]</code> (associative array)
<code>arr.map(fn)</code>	<code>array_map(\$fn, \$arr)</code>
<code>`Hi \${name}`</code>	<code>"Hi \$name"</code>
<code>a ?? b, a?.b</code>	<code>\$a ?? \$b, \$a?->b</code> (giống hệt)
<code>JSON.stringify / parse</code>	<code>json_encode / json_decode</code>
<code>npm, package.json</code>	<code>composer, composer.json</code>
Async mặc định (event loop)	Đồng bộ mặc định

Việc cần làm (JS thuần, chạy bằng Node hoặc console trình duyệt):

- Chứng minh `[] === []` là `false`, và giải thích được vì sao điều đó khiến React render lại
- Cho mảng 5 job, viết: lọc job có `salary > 100`, map lấy title, sắp xếp A → Z — **không được sửa mảng gốc** (kiểm tra bằng cách in mảng gốc sau đó)
- Viết hàm cập nhật 1 job trong mảng theo id, dùng map + spread, giữ nguyên bất biến
- Sửa 1 field lồng 2 cấp (`job.company.name`) mà không mutate object gốc
- Viết async function `getJobs()` dùng fetch tới `json-server`, có kiểm tra `res.ok` và try/catch
- Viết ví dụ closure của riêng bạn, giải thích được "stale closure" là gì

React hoạt động thế nào

Trước khi viết component, phải hiểu React thực sự làm gì. React **không** phải framework đầy đủ như NestJS/Laravel — nó chỉ là **thư viện dựng giao diện**, với một ý tưởng trung tâm: **UI = f(state)**. Bạn mô tả "với state này thì giao diện trông thế nào", React lo phần cập nhật DOM.

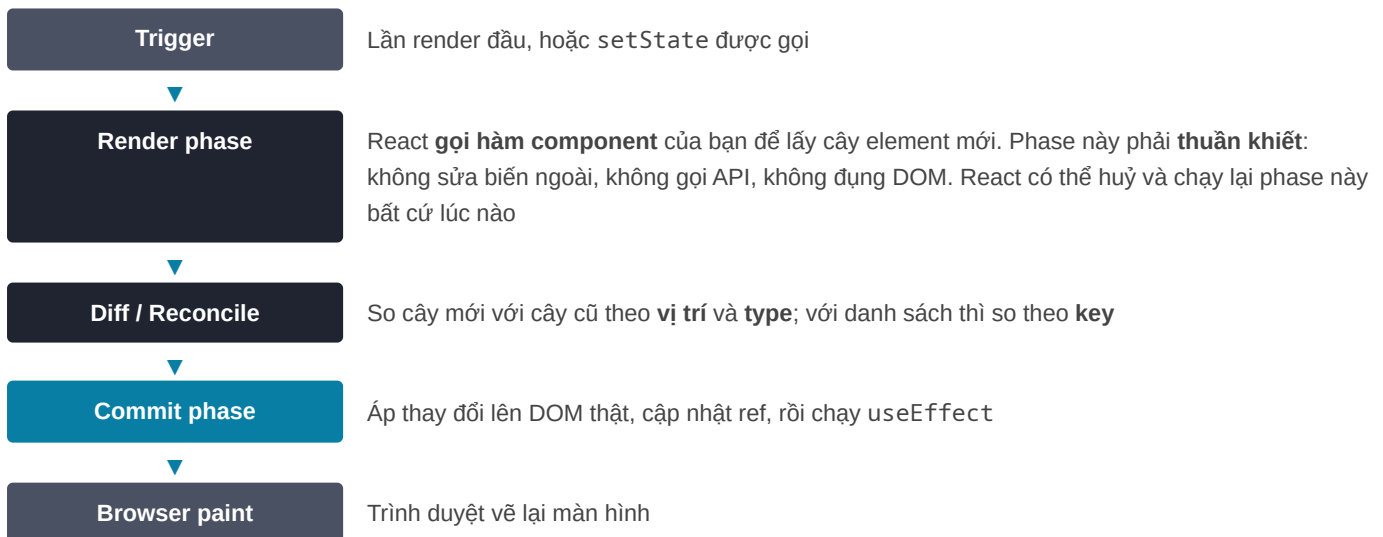
0.1 Declarative vs Imperative

```
// Imperative (DOM thuần) – bạn ra lệnh từng bước
const el = document.createElement('div');
el.textContent = count;
document.body.appendChild(el);
// và mỗi lần count đổi, bạn phải TỰ tìm đúng node để sửa

// Declarative (React) – bạn mô tả kết quả mong muốn
return <div>{count}</div>;
// count đổi → React tự tính ra cần sửa gì trên DOM
```

0.2 Virtual DOM & Reconciliation

Mỗi lần render, component của bạn trả về một **cây object mô tả UI** (React element — chính là Virtual DOM). React so sánh cây mới với cây của lần render trước (**reconciliation**), tính ra **tập thay đổi nhỏ nhất**, rồi mới dựng vào DOM thật. DOM thật rất chậm; so sánh object trong RAM thì nhanh — đó là toàn bộ lý do Virtual DOM tồn tại.



Hệ quả cần nhớ: "render" **không** đồng nghĩa với "cập nhật DOM". React có thể gọi lại hàm component của bạn nhiều lần mà DOM không đổi gì cả. Vì vậy: **không bao giờ đặt side-effect** (gọi API, ghi localStorage, sửa biến toàn cục) trực tiếp trong thân component.

0.3 Luồng dữ liệu một chiều

Dữ liệu chảy **từ cha xuống con** qua props. Con **không được sửa** props. Muốn con tác động lên cha thì cha truyền xuống một **hàm callback**. Nhờ ràng buộc này, khi UI sai bạn luôn truy ngược được nguồn dữ liệu — khác hẳn kiểu two-way binding dễ rối.

0.4 StrictMode: vì sao code chạy 2 lần

```
// src/main.jsx
<StrictMode>
  <App />
</StrictMode>
```

Ở **chế độ development**, StrictMode cố ý **gọi component 2 lần** và **chạy effect 2 lần** (mount → unmount → mount lại) để phát hiện sớm: component không thuần khiết, effect thiếu cleanup. Ở **production nó không làm vậy**. Thấy log in 2 lần là bình thường — **đừng gỡ StrictMode**, hãy sửa code cho đúng.

0.5 React 19 & hệ sinh thái hiện tại

Thành phần	Vai trò
<code>react</code>	Lõi: component, hook, element — không biết gì về trình duyệt
<code>react-dom</code>	Renderer cho web (<code>createRoot</code> , các hook liên quan DOM/form)
Vite	Dev server + bundler (thay thế Create React App đã ngừng phát triển)
React Router	Điều hướng nhiều trang (Part 09)
React Compiler v1.0	Trình biên dịch tự memo hoá — giảm nhu cầu <code>useMemo</code> / <code>useCallback</code> viết tay (Part 10)

Việc cần làm:

1. Chưa code — vẽ lại sơ đồ mục 0.2 ra giấy bằng ngôn từ của bạn
2. Trả lời: vì sao React cần "key" khi render danh sách (gợi ý: liên quan tới bước Diff)
3. Trả lời: "render" và "cập nhật DOM" khác nhau chỗ nào
4. Mở app hiện có, thêm `console.log` trong thân một component rồi quan sát số lần in ra — giải thích bằng StrictMode

Khởi tạo dự án với Vite & cấu trúc thư mục

Mục tiêu: Hiểu Vite làm gì, mỗi file trong dự án đóng vai trò gì, và cách tổ chức thư mục để dự án lớn không loạn.

1.1 Tạo dự án

```
npm create vite@latest react-crash-course -- --template react
cd react-crash-course
npm install
npm run dev           # → http://localhost:5173
```

Script	Tác dụng
<code>npm run dev</code>	Dev server, hot reload (HMR) — giữ nguyên state khi sửa code
<code>npm run build</code>	Build production vào <code>dist/</code> (minify, tách chunk, hash tên file)
<code>npm run preview</code>	Chạy thử bản build — kiểm tra trước khi deploy
<code>npm run lint</code>	ESLint — bắt lỗi hook sai quy tắc

Vì sao Vite nhanh: ở dev, Vite **không bundle** — nó phục vụ file dưới dạng ES module gốc để trình duyệt tự nạp, nên khởi động gần như tức thì bất kể dự án lớn cỡ nào. Chỉ khi build nó mới bundle (bằng Rollup). Create React App (webpack) bundle lại toàn bộ mỗi lần khởi động — chậm hơn nhiều, và đã ngừng phát triển.

1.2 Điểm khởi đầu

```
// index.html – Vite dùng file này làm entry (không phải file JS)
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>
```

```
// src/main.jsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';
import './index.css';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

Toàn bộ app React của bạn sống bên trong **một thẻ div duy nhất** — đó chính là ý nghĩa của "Single Page Application".

1.3 Cấu trúc thư mục nên theo

Thư mục	Chứa gì
<code>src/components/</code>	Component tái sử dụng, không gắn với route (Card, Navbar, Spinner)
<code>src/pages/</code>	Component tương ứng một route (HomePage, JobPage)
<code>src/layouts/</code>	Khung chung bọc nhiều trang (MainLayout: Navbar + <code><Outlet /></code>)
<code>src/hooks/</code>	Custom hook (<code>useJobs</code> , <code>useFetch</code>) — Part 07
<code>src/services/</code> hoặc <code>api/</code>	Hàm gọi API — tách khỏi component
<code>src/context/</code>	Context provider — Part 08
<code>src/assets/</code>	Ảnh, icon (Vite tự hash khi build)

Quy ước đặt tên: file component viết **PascalCase** (`JobCard.jsx`), hook bắt đầu bằng **use** (`useJobs.js`), hằng số viết UPPER_SNAKE. Nhất quán quan trọng hơn "chọn đúng" — chọn một kiểu rồi giữ nguyên.

1.4 Biến môi trường

```
# .env — CHỈ biến có tiền tố VITE_ mới được lộ ra client
VITE_API_URL=http://localhost:8000/api/v1
```

```
const baseUrl = import.meta.env.VITE_API_URL;
```

Cảnh báo bảo mật: mọi biến `VITE_*` đều bị **nhúng thẳng vào file JS** gửi xuống trình duyệt — ai cũng xem được. **Không bao giờ** đặt secret key, DB password, API secret vào đây. Client chỉ được biết những gì công khai được.

Việc cần làm:

1. Tạo dự án Vite mới (hoặc dùng lại dự án job listings đang có)
2. Sắp xếp lại thư mục theo bảng 1.3 — tách `components/`, `pages/`, `layouts/`
3. Đọc kỹ `index.html` + `main.jsx`, giải thích được app "gắn" vào DOM chỗ nào
4. Tạo `.env` với `VITE_API_URL`, đọc ra và log — xác nhận nó xuất hiện trong bundle sau khi build (mở file trong `dist/` tìm chuỗi đó)
5. Chạy `npm run build` + `npm run preview`, xem kích thước bundle

JSX, Component & Props

Mục tiêu: Viết component đúng chuẩn, hiểu JSX thực chất là gì, và nắm quy tắc truyền props.

2.1 JSX thực chất là gì

```
// Bạn viết:  
const el = <h1 className="title">Xin chào</h1>;  
  
// Vite/Babel biên dịch thành lời gọi hàm (React 17+ dùng jsx runtime):  
const el = jsx('h1', { className: 'title', children: 'Xin chào' });
```

Nghĩa là JSX chỉ là **cú pháp đường** để tạo object mô tả UI. Vì cuối cùng là JavaScript, nên có các quy tắc:

Quy tắc	Vì sao
<code>className</code> thay <code>class</code>	<code>class</code> là từ khoá của JS
<code>htmlFor</code> thay <code>for</code>	Tương tự
Thuộc tính viết camelCase: <code>onClick</code> , <code>tabIndex</code>	Đang là thuộc tính object JS
Phải đóng mọi thẻ: <code></code> , <code>
</code>	JSX nghiêm ngặt hơn HTML
Chỉ trả về một node gốc	Một hàm chỉ return được 1 giá trị → dùng <code><>...</></code> (Fragment)
<code>{ }</code> để nhúng biểu thức	Chỉ nhận biểu thức , không nhận <code>if</code> / <code>for</code>

2.2 Component

```
// Component = HÀM trả về JSX. Tên PHẢI viết hoa chữ đầu.  
export default function JobCard({ title, type, salary }) {  
  return (  
    <div className="rounded-xl bg-white p-4 shadow">  
      <h3>{title}</h3>  
      <p>{type}</p>  
      <p>{salary ?? 'Thoả thuận'}</p>  
    </div>  
  );  
}
```

Vì sao tên component phải viết hoa: JSX quy ước chữ thường (`<div>`) là thẻ HTML, chữ hoa (`<JobCard>`) là component. Đặt tên `jobCard` thì React sẽ đi tìm thẻ HTML tên "jobcard" và render ra **trang trắng** không báo lỗi rõ ràng.

2.3 Props

```
// Cha truyền xuống
<JobCard title={job.title} type={job.type} salary={job.salary} />
<JobCard {...job} /> // spread khi tên trùng khớp

// Con nhận – destructure ngay ở tham số, có giá trị mặc định
function JobCard({ title, type = 'Full-Time' }) { }

// children: nội dung đặt giữa 2 thẻ
function Card({ children }) { return <div className="card">{children}</div>; }
<Card><h3>Tiêu đề</h3></Card>
```

Props là read-only. Component **không được** sửa props nhận vào (`props.title = 'x'` là sai). Nếu cần thay đổi dữ liệu → dữ liệu đó phải là **state** ở component cha, và cha truyền xuống hàm để con gọi ngược lên.

2.4 Truyền hàm xuống con

```
// Cha
<JobCard job={job} onDelete={() => handleDelete(job.id)} />

// Con
<button onClick={onDelete}>Xoá</button> // ✓ truyền tham chiếu hàm
<button onClick={onDelete()}>Xoá</button> // ✗ GỌI LUÔN khi render!
```

Việc cần làm:

1. Tách trang chủ thành các component nhỏ: Navbar, Hero, HomeCards, JobListings, JobListing
2. Viết Card nhận children + prop bg có giá trị mặc định
3. Truyền dữ liệu job xuống bằng props, thử cả cách liệt kê từng prop và cách `{...job}`
4. Cố tình đặt tên component chữ thường để thấy trang trắng — hiểu vì sao
5. Cố tình viết `onClick={handleDelete() }` để thấy hàm chạy ngay khi render

State với useState — tư duy bất biến

Mục tiêu: Hiểu state là gì, vì sao phải cập nhật bất biến, và 3 hiểu lầm phổ biến nhất về setState.

3.1 useState cơ bản

```
const [count, setCount] = useState(0);  
//   ▲ giá trị hiện tại   ▲ hàm cập nhật   ▲ giá trị khởi tạo (chỉ dùng ở lần render ĐẦU)  
  
setCount(count + 1);           // yêu cầu React render lại với giá trị mới
```

Quy tắc Hook (bắt buộc): chỉ gọi hook ở **cấp cao nhất** của component — không đặt trong if, vòng lặp, hàm lồng. Lý do: React nhận diện hook theo **thứ tự gọi**; đổi thứ tự giữa các lần render là hỏng toàn bộ state. ESLint plugin react-hooks sẽ bắt lỗi này.

3.2 Hiểu lầm 1: "setState cập nhật ngay lập tức"

```
function handleClick() {  
  setCount(count + 1);  
  console.log(count);    // ✗ vẫn in ra giá trị CŨ  
}
```

Vì count là **hằng số của lần render này** (closure — xem Part JS). setCount chỉ **lên lịch** một lần render mới; giá trị mới chỉ xuất hiện ở **lần render sau**. Đây gọi là "state là một snapshot".

3.3 Hiểu lầm 2: gọi setState nhiều lần thì cộng dồn

```
setCount(count + 1);  
setCount(count + 1);  
setCount(count + 1);           // count = 0 → kết quả là 1, KHÔNG phải 3  
  
// ✓ Dùng dạng hàm cập nhật (updater) khi giá trị mới phụ thuộc giá trị cũ  
setCount((prev) => prev + 1);  
setCount((prev) => prev + 1);  
setCount((prev) => prev + 1);    // → 3 ✓
```

React **gộp (batch)** các lần setState trong cùng một sự kiện thành **một lần render** — đó là tối ưu, không phải bug. Dạng updater đọc giá trị mới nhất trong hàng đợi nên luôn đúng.

3.4 Hiểu lầm 3: sửa trực tiếp object/array trong state

```
// ✗ SAI – mutate: React so sánh thấy CÙNG tham chiếu → không render lại
jobs.push(newJob);
setJobs(jobs);
form.title = 'X';
setForm(form);

// ✓ ĐÚNG – luôn tạo object/array MỚI
setJobs([...jobs, newJob]); // thêm
setJobs(jobs.filter(j => j.id !== id)); // xoá
setJobs(jobs.map(j => j.id === id ? { ...j, ...patch } : j)); // sửa
setForm({ ...form, title: 'X' }); // sửa 1 field
```

3.5 Chọn cấu trúc state cho tốt

- **Đừng lưu thứ tính được ra state.** Có jobs và keyword thì filteredJobs chỉ cần tính khi render, không cần state riêng (state thừa = nguồn dữ liệu lệch nhau).
- **Đừng để state trùng lặp.** Lưu selectedId thay vì lưu cả object selectedJob đã copy.
- **Nhóm state đi cùng nhau.** Form nhiều field → một object, hoặc dùng useReducer (Part 08).
- **Đặt state ở đúng độ cao.** Càng gần nơi dùng càng tốt; chỉ nâng lên cha khi có 2 component cùng cần (Part 08).

State vs biến thường: biến thường (`let x = 0`) bị **reset** sau mỗi lần render và **không** kích hoạt render. State thì được React **giữ lại giữa các lần render** và **kích hoạt render khi đổi**. Cần giữ giá trị mà **không** muốn render lại (id của timer, tham chiếu DOM) → dùng useRef.

Việc cần làm:

1. Viết bộ đếm, chứng minh `setCount(count+1)` ×3 chỉ tăng 1, còn dạng updater tăng 3
2. Log count ngay sau `setCount` để thấy giá trị cũ — giải thích bằng "snapshot"
3. Làm ô tìm kiếm lọc danh sách job: **không** tạo state `filteredJobs`, chỉ tính khi render
4. Viết 3 thao tác thêm / sửa / xoá job trong state theo đúng cách bất biến
5. Cố tình mutate (`jobs.push`) để tận mắt thấy UI không cập nhật
6. Làm form nhiều field bằng **một** object state + một handler `handleChange` dùng [`e.target.name`]

Render danh sách, key & render có điều kiện

Mục tiêu: Render danh sách đúng cách và hiểu **thật sự** key dùng để làm gì — đây là chỗ gây bug âm thầm nhiều nhất với người mới.

4.1 map để render danh sách

```
<div>
  {jobs.map((job) => (
    <JobListing key={job.id} job={job} />
  ))}
</div>
```

4.2 Key dùng để làm gì

Key **không phải** để "React biết thứ tự". Key là **danh tính** của phần tử: nó cho React biết "element này ở lần render trước và lần này là **cùng một thứ**". Nhờ đó React biết nên **giữ lại** DOM node cùng toàn bộ **state bên trong** component đó, hay **huỷ đi tạo mới**.

Cách đặt key	Đánh giá
<code>key={job.id}</code>	✓ Đúng — id ổn định từ dữ liệu
<code>key={index}</code>	✗ Nguy hiểm khi danh sách có thể sắp xếp / thêm / xoá ở giữa
<code>key={Math.random()}</code>	✗ Rất tệ — key đổi mỗi lần render → React huỷ và dựng lại toàn bộ, mất state, mất focus, chậm
Không có key	React cảnh báo trong console và mặc định dùng index

Bug thật khi dùng index làm key: danh sách có ô input; bạn gõ vào ô thứ 2 rồi xoá phần tử thứ 1. Vì key là index, React tưởng phần tử 2 "chính là" phần tử 1 cũ → **giữ nguyên state của ô cũ** → nội dung bạn gõ nhảy sang dòng khác. Đây là loại bug rất khó truy vết nếu không hiểu key.

4.3 Render có điều kiện

```
{ loading && <Spinner /> } // hiện khi true
{ error ? <Error msg={error} /> : <JobList jobs={jobs} /> }

{ jobs.length > 0
  ? jobs.map((j) => <JobListing key={j.id} job={j} />)
  : <p>Chưa có công việc nào</p> }

// Logic phức tạp → tách ra biến trước khi return, đừng nhồi vào JSX
let content;
if (loading) content = <Spinner />;
else if (error) content = <Error msg={error} />;
else content = <JobList jobs={jobs} />;

return <div>{content}</div>;
```

Nhắc lại bẫy từ Part JS: `{jobs.length && ...}` in ra số 0 khi mảng rỗng. Luôn viết `{jobs.length > 0 && ...}`. Ngoài ra `null`, `undefined`, `false` đều không render gì — nên `return null` là cách hợp lệ để "không hiện gì".

4.4 Key để chủ động reset state

```
// Đổi key → React hủy component cũ, tạo mới hoàn toàn (state về khởi tạo)
<JobForm key={selectedJobId} job={selectedJob} />
```

Đây là mẹo chính thống được React khuyến nghị: thay vì viết `useEffect` để "reset form khi đổi job", chỉ cần đổi `key`.

Việc cần làm:

1. Render danh sách job bằng `map` + `key` là `id`
2. Tái hiện bug `key=index`: danh sách có input, xóa phần tử đầu, quan sát giá trị input nhảy sai; sau đó sửa bằng `key=id`
3. Xử lý 3 trạng thái `loading` / `error` / `empty` cho danh sách
4. Tạo bug `{jobs.length && ...}` hiện số 0 rồi sửa lại
5. Dùng mẹo `key` ở mục 4.4 để reset form khi chọn job khác

Sự kiện & Form (controlled component)

Mục tiêu: Xử lý sự kiện đúng cách và làm form theo mô hình controlled — nền tảng cho mọi trang thêm/sửa dữ liệu.

5.1 Sự kiện

```
<button onClick={handleClick}>Lưu</button> // truyền tham chiếu
<button onClick={() => handleDelete(job.id)}>Xóa</button> // cần tham số → bọc arrow

function handleSubmit(e) {
  e.preventDefault(); // chặn browser reload trang khi submit form
}
```

React dùng **synthetic event** — một lớp bọc chuẩn hoá sự kiện DOM để hoạt động giống nhau trên mọi trình duyệt. API dùng gần như y hệt DOM thật (`e.target`, `e.preventDefault()`, `e.stopPropagation()`).

5.2 Controlled component

```
const [title, setTitle] = useState('');

<input
  value={title} // giá trị LUÔN lấy từ state
  onChange={(e) => setTitle(e.target.value)} // gõ → cập nhật state → render lại
/>
```

"Controlled" nghĩa là **React nắm quyền kiểm soát** giá trị của input, DOM chỉ hiển thị lại state. Nhờ vậy bạn validate, format, disable nút submit... đều dễ dàng vì luôn có dữ liệu trong tay.

Lỗi thường gặp: đặt `value={title}` mà **quên** `onChange` → input bị "đơ", gõ không lên chữ, kèm cảnh báo trong console. Nếu thực sự muốn không kiểm soát thì dùng `defaultValue`.

5.3 Form nhiều field — một handler duy nhất

```
const [form, setForm] = useState({ title: '', type: 'Full-Time', salary: '' });

function handleChange(e) {
  const { name, value, type, checked } = e.target;

  setForm((prev) => ({
    ...prev,
    [name]: type === 'checkbox' ? checked : value, // computed key
  }));
}

<input name="title" value={form.title} onChange={handleChange} />
<input name="salary" value={form.salary} onChange={handleChange} />
```

5.4 Submit

```
async function handleSubmit(e) {
  e.preventDefault();

  if (!form.title.trim()) {
    setError('Tiêu đề là bắt buộc');
    return;
  }

  setSubmitting(true);
  try {
    await createJob(form);
    navigate('/jobs');
  } catch (err) {
    setError(err.message);
  } finally {
    setSubmitting(false); // finally: chạy cả khi lỗi
  }
}

<button type="submit" disabled={submitting}>
  {submitting ? 'Đang lưu...' : 'Lưu'}
</button>
```

Luôn khoá nút khi đang submit. Không khoá thì user bấm nhiều lần → tạo trùng bản ghi. React 19 có `useActionState` + `useFormStatus` làm việc này gọn hơn (Part 11).

Việc cần làm:

1. Làm trang `AddJobPage` với form controlled đầy đủ field
2. Dùng **một** state object + **một** `handleChange` cho mọi input
3. Validate phía client (bắt buộc nhập title, lương phải là số), hiện lỗi ngay dưới field
4. Khoá nút submit + đổi chữ khi đang gửi
5. Làm trang `EditJobPage`: đổ dữ liệu cũ vào form rồi cập nhật
6. Bỏ `e.preventDefault()` một lần để thấy trang reload — hiểu vì sao cần nó

useEffect — dùng đúng và dùng ít

Mục tiêu: Hiểu Effect thực sự dùng để làm gì (đồng bộ với hệ thống **bên ngoài** React), và nhận ra phần lớn useEffect người mới viết là **không cần thiết**.

6.1 Cú pháp & 3 dạng dependency

```
useEffect(() => {
  // side effect: gọi API, subscribe, timer, đọc/ghi localStorage...

  return () => {
    // CLEANUP: chạy trước lần effect kế tiếp và khi component unmount
  };
}, [deps]);
```

Dependency	Chạy khi nào
<code>[]</code>	Chỉ sau lần render đầu (mount). Dev + StrictMode: chạy 2 lần
<code>[a, b]</code>	Sau mount, và mỗi khi <code>a</code> hoặc <code>b</code> đổi tham chiếu/giá trị
không truyền	Sau MỌI lần render — hầu như luôn là lỗi

6.2 Cleanup — bắt buộc, không phải tùy chọn

```
useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id); // không có dòng này = rò rỉ timer
}, []);

useEffect(() => {
  window.addEventListener('resize', onResize);
  return () => window.removeEventListener('resize', onResize);
}, []);
```

StrictMode cố tình mount → unmount → mount lại ở dev chính là để **ép bạn phát hiện thiếu cleanup**. Nếu bật app thấy có 2 timer chạy, đó là effect của bạn chưa dọn dẹp.

6.3 Bẫy dependency: vòng lặp vô hạn

```
// ✗ Object/array tạo mới mỗi lần render → deps LUÔN khác → effect chạy mãi
useEffect(() => { fetchJobs(filters); }, [filters]); // filters = {} inline ở cha

// ✗ setState không điều kiện trong effect phụ thuộc chính state đó
useEffect(() => { setCount(count + 1); }, [count]);

// ✓ Dùng giá trị primitive làm dependency
useEffect(() => { fetchJobs(type, keyword); }, [type, keyword]);
```

Đừng bao giờ "im lặng" ESLint bằng cách xoá bớt dependency hoặc thêm `// eslint-disable-next-line react-hooks/exhaustive-deps`. Cảnh báo đó gần như luôn đúng — nó đang chỉ ra rằng effect của bạn phụ thuộc thứ mà bạn chưa xử lý. Cách sửa đúng: chuyển hàm vào trong effect, dùng dạng updater của `setState`, hoặc dùng `useEffectEvent` (mục 6.5).

6.4 Khi nào KHÔNG cần `useEffect`

Tình huống	Cách đúng
Tính giá trị dẫn xuất từ props/state (lọc, tổng, format)	Tính thẳng khi render — không cần effect + state
Cập nhật state khi props đổi	Tính khi render, hoặc đổi <code>key</code> để reset (Part 04)
Xử lý sau khi user bấm nút	Viết trong event handler , không phải effect
Gửi request khi submit form	Trong <code>handleSubmit</code>
Fetch dữ liệu cho trang	Effect được — hoặc tốt hơn: loader của Router (Part 09) / thư viện data fetching
Đồng bộ với hệ thống ngoài: timer, websocket, listener, thư viện DOM bên thứ ba	Đây mới là việc của Effect

Câu hỏi tự kiểm tra trước khi viết `useEffect`: "Việc này có phải là **đồng bộ với thứ bên ngoài React** không?"
Nếu không — 90% là bạn không cần Effect.

6.5 `useEffectEvent` (React 19.2)

```
import { useEffectEvent } from 'react';

function ChatRoom({ roomId, theme }) {
  // Tách phần "sự kiện" ra: đọc được theme MỚI NHẤT nhưng KHÔNG làm effect chạy lại
  const onConnected = useEffectEvent(() => {
    showNotification('Đã kết nối!', theme);
  });

  useEffect(() => {
    const conn = createConnection(roomId);
    conn.on('connected', onConnected);
    conn.connect();
    return () => conn.disconnect();
  }, [roomId]); // ✓ KHÔNG đưa onConnected vào deps (theo đúng quy định)
}
```

Trước 19.2, đổi theme sẽ khiến effect chạy lại → ngắt và kết nối lại phòng chat một cách vô lý.
`useEffectEvent` giải quyết đúng bài toán đó. Cần `eslint-plugin-react-hooks` bản mới để lint đúng.

Việc cần làm:

1. Viết effect fetch danh sách job khi mount, có cleanup
2. Cố tình bỏ [] để thấy vòng lặp fetch vô hạn (mở tab Network), rồi sửa
3. Viết effect có `setInterval`, quan sát ở StrictMode chạy 2 timer khi thiếu cleanup
4. Rà lại toàn bộ code cũ: tìm những `useEffect` chỉ để tính giá trị dẫn xuất và **xoá** chúng đi
5. Viết effect có dependency là object inline để tái hiện vòng lặp, rồi sửa bằng dependency primitive
6. Thử `useEffectEvent` cho một effect cần đọc state mới nhất mà không muốn chạy lại

Gọi API & Custom Hook

Mục tiêu: Fetch dữ liệu đúng chuẩn (loading / error / race condition), rồi đóng gói lại thành custom hook tái sử dụng.

7.1 Mẫu fetch đầy đủ

```
const [jobs, setJobs] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  const controller = new AbortController(); // để huỷ request cũ

  async function load() { // khai hàm async BÊN TRONG effect
    try {
      setLoading(true);
      setError(null);

      const res = await fetch(url, { signal: controller.signal });
      if (!res.ok) throw new Error(`HTTP ${res.status}`);

      setJobs(await res.json());
    } catch (err) {
      if (err.name !== 'AbortError') setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  load();
  return () => controller.abort(); // cleanup: huỷ request khi deps đổi/unmount
}, [url]);
```

Hai lỗi bắt buộc phải tránh:

- (1) **Không** viết `useEffect(async () => {...})` — hàm async trả về Promise, còn React mong nhận về **hàm cleanup**. Phải khai hàm async bên trong rồi gọi.
- (2) **Race condition**: user gõ tìm kiếm nhanh → nhiều request bay đi; response về **không theo thứ tự** → kết quả cũ ghi đè kết quả mới. `AbortController` trong cleanup giải quyết triệt để.

7.2 Tách tầng gọi API ra khỏi component

```
// src/services/jobService.js — component KHÔNG nên biết chi tiết URL/header
const BASE = import.meta.env.VITE_API_URL;

async function request(path, options = {}) {
  const res = await fetch(`${BASE}${path}`, {
    headers: { 'Content-Type': 'application/json', 'Accept': 'application/json' },
    ...options,
  });

  if (!res.ok) throw new Error((await res.json()).catch(() => ({}))).message ?? 'Request failed');
  return res.status === 204 ? null : res.json();
}

export const getJobs = () => request('/jobs');
export const getJob = (id) => request(`/jobs/${id}`);
export const createJob = (data) => request('/jobs', { method: 'POST', body: JSON.stringify(data) });
export const deleteJob = (id) => request(`/jobs/${id}`, { method: 'DELETE' });
```

7.3 Custom Hook — tái sử dụng logic có state

```
// src/hooks/useFetch.js — tên PHẢI bắt đầu bằng "use"
export function useFetch(fetcher, deps = []) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let alive = true;

    setLoading(true);
    fetcher()
      .then((d) => alive && setData(d))
      .catch((e) => alive && setError(e.message))
      .finally(() => alive && setLoading(false));

    return () => { alive = false; }; // chặn setState sau khi unmount
  }, deps);

  return { data, loading, error };
}

// Dùng
const { data: jobs, loading, error } = useFetch(getJobs, []);
```

Custom hook chia sẻ LOGIC, không chia sẻ STATE. Hai component cùng gọi useFetch sẽ có hai bộ state riêng biệt. Muốn dùng chung một dữ liệu thì phải nâng state lên cha hoặc dùng Context (Part 08).

7.4 Khi nào nên dùng thư viện

Tự viết fetch là cách tốt nhất để học. Nhưng dự án thật sẽ cần cache, tự refetch, chống trùng request, retry, invalidate sau khi mutate — lúc đó dùng **TanStack Query** hoặc **SWR** thay vì tự viết lại. Hãy tự viết ở crash course này trước, rồi mới chuyển sang thư viện để thấy nó giải quyết vấn đề gì.

Việc cần làm:

1. Tạo `src/services/jobService.js`, chuyển toàn bộ lời gọi `fetch` rải rác trong component về đây
2. Viết mẫu `fetch` đầy đủ ở 7.1 cho trang danh sách (loading / error / dữ liệu rỗng)
3. Tái hiện race condition: ô tìm kiếm gõ nhanh, log kết quả nhận về theo thứ tự, rồi sửa bằng `AbortController`
4. Đóng gói thành `useFetch` và dùng lại ở ít nhất 2 trang
5. Thử tắt `json-server` để test nhánh lỗi hiển thị đúng

Chia sẻ state: lifting up, Context, useReducer

Mục tiêu: Biết chọn đúng công cụ khi nhiều component cần cùng một dữ liệu — và biết khi nào **không** nên dùng Context.

8.1 Lifting state up (nâng state lên cha)

Hai component cần cùng một dữ liệu → đưa state lên **component cha gần nhất** của cả hai, rồi truyền xuống bằng props. Đây là cách **mặc định** và nên là lựa chọn đầu tiên.

```
function JobsPage() {
  const [keyword, setKeyword] = useState(''); // state ở cha

  return (
    <>
      <SearchBar value={keyword} onChange={setKeyword} />
      <JobList keyword={keyword} />
    </>
  );
}
```

8.2 Context — khi props phải xuyên quá nhiều tầng

Vấn đề "props drilling": truyền user từ App → Layout → Navbar → UserMenu, trong đó Layout và Navbar **không dùng** mà chỉ chuyển tiếp. Context giải quyết đúng chuyện đó.

```
// src/context/AuthContext.jsx
import { createContext, useContext, useState } from 'react';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(() => localStorage.getItem('token'));

  // React 19: dùng thẳng <AuthContext> làm provider, không cần .Provider
  return (
    <AuthContext value={{ user, token, setUser, setToken }}>
      {children}
    </AuthContext>
  );
}

// Hook tiện dụng + báo lỗi rõ ràng nếu dùng ngoài Provider
export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth phải dùng bên trong <AuthProvider>');
  return ctx;
}
```

Context không phải state manager và không tối ưu hiệu năng. Khi giá trị context đổi, **mọi** component đang useContext đều render lại. Vì vậy: (1) đừng nhồi mọi thứ vào một context khổng lồ — tách theo mỗi quan tâm (Auth / Theme / Cart); (2) đừng dùng Context cho dữ liệu đổi liên tục (giá trị input đang gõ); (3) dữ liệu từ server thì nên dùng thư viện data fetching thay vì Context.

8.3 useReducer — khi state có nhiều nhánh chuyển đổi

```
function jobsReducer(state, action) {
  switch (action.type) {
    case 'loading': return { ...state, loading: true, error: null };
    case 'success': return { loading: false, error: null, items: action.payload };
    case 'error':    return { ...state, loading: false, error: action.error };
    case 'remove':  return { ...state, items: state.items.filter((j) => j.id !== action.id) };
    default:        throw new Error(`Unknown action: ${action.type}`);
  }
}

const [state, dispatch] = useReducer(jobsReducer, { items: [], loading: true, error: null });

dispatch({ type: 'success', payload: jobs });
```

Dùng khi	Chọn
1–3 giá trị đơn giản, độc lập	<code>useState</code>
Nhiều giá trị liên quan nhau, nhiều cách chuyển trạng thái	<code>useReducer</code>
Dữ liệu cần cho nhiều nhánh cây component	<code>Context</code> (+ reducer nếu phức tạp)
Dữ liệu từ server, cần cache/refetch	TanStack Query / SWR
State toàn cục lớn, phức tạp	Zustand / Redux Toolkit

Việc cần làm:

1. Nâng state ô tìm kiếm lên cha, chia sẻ giữa SearchBar và JobList
2. Tạo AuthContext + useAuth (chưa cần API thật, lưu token giả vào localStorage)
3. Bọc AuthProvider ở App, đọc user trong Navbar bằng useAuth — xoá hết props drilling
4. Cố tình gọi useAuth ở component ngoài Provider để thấy lỗi rõ ràng bạn tự throw
5. Chuyển state loading/error/items của trang danh sách sang useReducer, so sánh xem code dễ đọc hơn không

React Router — routing, loader & form action

Mục tiêu: Làm app nhiều trang thật sự: URL đổi, nested layout, lấy tham số, và tải dữ liệu bằng **loader** thay vì `useEffect`.

Về phiên bản: dự án của bạn dùng `react-router-dom@7`. Từ v7, các package được gộp về một tên duy nhất là `react-router` (`react-router-dom` vẫn còn để tương thích ngược). Bản mới nhất hiện tại là v8 — API 3 mode dưới đây **không đổi**, nên kiến thức phần này dùng được cho cả hai.

9.1 Ba "mode" của React Router

Mode	API chính	Khi nào dùng
Declarative	<code><BrowserRouter></code> , <code><Routes></code> , <code><Route></code> , <code>useNavigate</code>	App nhỏ, chỉ cần đổi URL
Data	<code>createBrowserRouter</code> , <code>RouterProvider</code> , <code>loader</code> , <code>action</code> , <code>useFetcher</code>	Dự án của bạn đang ở mode này — có tải dữ liệu theo route
Framework	Vite plugin, route module, type-safe params, SSR/SSG	Cần server-side rendering, code splitting tự động

9.2 Cấu hình router (Data mode)

```
import { createBrowserRouter, RouterProvider } from 'react-router-dom';

const router = createBrowserRouter([
  {
    path: '/',
    element: <MainLayout />,           // layout bọc mọi route con
    errorElement: <ErrorPage />,       // hiện khi loader lỗi / route không tồn tại
    children: [
      { index: true, element: <HomePage /> },           // path '/'
      { path: 'jobs', element: <JobsPage /> },
      { path: 'jobs/:id', element: <JobPage />, loader: jobLoader },
      { path: 'add-job', element: <AddJobPage /> },
      { path: '*', element: <NotFoundPage /> },          // catch-all
    ],
  },
]);

export default function App() { return <RouterProvider router={router} />; }
```

```
// MainLayout – <Outlet /> là chỗ route con được render vào
import { Outlet } from 'react-router-dom';

export default function MainLayout() {
  return (
    <>
      <Navbar />
      <main><Outlet /></main>
      <Toaster />
    </>
  );
}
```

9.3 Điều hướng

```
// ✓ Link: điều hướng client-side, KHÔNG tải lại trang
<Link to={`/jobs/${job.id}`}>Xem chi tiết</Link>

// NavLink: tự biết mình có đang active không → tô sáng menu
<NavLink to="/jobs" className={({ isActive }) => isActive ? 'text-white' : 'text-gray-400'}>
  Việc làm
</NavLink>

// Điều hướng bằng code (sau khi submit thành công)
const navigate = useNavigate();
navigate('/jobs');
navigate(-1); // quay lại như nút Back
navigate('/jobs', { replace: true }); // thay thế history, không back về được

// ✗ <a href="/jobs"> – tải lại toàn bộ app, mất hết state
```

9.4 Lấy tham số & loader

```
// Đọc :id từ URL
const { id } = useParams();

// Đọc query string ?type=Full-Time
const [searchParams, setSearchParams] = useSearchParams();
const type = searchParams.get('type');
```

```
// Loader: Router gọi TRƯỚC khi render component → không còn màn hình trắng nhấp nháy
export async function jobLoader({ params, request }) {
  const res = await fetch(`/api/jobs/${params.id}`, { signal: request.signal });

  if (!res.ok) throw new Response('Not Found', { status: 404 }); // → errorElement

  return res.json();
}

// Trong component
const job = useLoaderData();
const { state } = useNavigation(); // 'idle' | 'loading' | 'submitting'
```

Vì sao loader tốt hơn useEffect để fetch: loader chạy **song song** với việc tải code của trang và **trước khi render**, nên tránh được "waterfall" (render → effect → fetch → render lại) và không bị hiện tượng nháy loading. Router cũng tự huỷ request khi user điều hướng đi (nhờ `request.signal`) — chính là `AbortController` ở Part 07, nhưng miễn phí.

Việc cần làm:

1. Dựng router đủ 6 route: home, jobs, job detail, add, edit, 404 — có `MainLayout + Outlet`
2. Thay mọi `<a href>` nội bộ bằng `Link`; dùng `NavLink` để tô sáng menu đang mở
3. Trang chi tiết: dùng `loader + useLoaderData` thay cho `useEffect`
4. Thêm `errorElement`, test bằng cách vào `/jobs/99999`
5. Lọc theo `?type=` bằng `useSearchParams` (bộ lọc lưu trên URL để chia sẻ link được)
6. Dùng `useNavigation()` hiện thành loading khi đang chuyển trang

Hiệu năng: re-render, memo & React Compiler

Mục tiêu: Hiểu vì sao component render lại, và tối ưu **đúng chỗ** — chứ không rải useMemo khắp nơi.

10.1 Khi nào một component render lại

- State của chính nó đổi
- Component **cha** render lại (mặc định con render theo, dù **props không đổi**)
- Context nó đang dùng đổi giá trị

Render lại không đồng nghĩa với chậm. React chỉ cập nhật DOM khi có khác biệt thật. Đừng tối ưu trước khi đo — dùng **React DevTools Profiler** hoặc **Performance Tracks** mới có ở React 19.2 (tab Performance của Chrome, có track "Scheduler" và "Components") để tìm đúng chỗ nghẽn.

10.2 Ba công cụ memo hoá

API	Ghi nhớ cái gì	Dùng khi
<code>useMemo(fn, deps)</code>	Giá trị trả về	Tính toán nặng, hoặc cần object/array giữ nguyên tham chiếu
<code>useCallback(fn, deps)</code>	Hàm	Truyền hàm xuống con đã bọc <code>memo</code> , hoặc làm dependency của effect
<code>memo(Component)</code>	Kết quả render của component	Component nặng, nhận props ít đổi

```
const filtered = useMemo(
  () => jobs.filter((j) => j.title.toLowerCase().includes(keyword.toLowerCase())),
  [jobs, keyword],
);

const handleDelete = useCallback((id) => deleteJob(id), []);

const JobListing = memo(function JobListing({ job, onDelete }) { /* ... */ });
```

Đừng lạm dụng: memo chỉ có tác dụng khi props **thật sự** ổn định. Bọc memo mà cha vẫn truyền xuống `{...}` hoặc `() => ...` tạo mới mỗi render thì **vô ích** — còn tốn thêm chi phí so sánh props.

10.3 React Compiler v1.0 — thay đổi cuộc chơi

React Compiler (bản 1.0 phát hành 10/2025) là trình biên dịch **tự động memo hoá**: nó phân tích code và tự chèn tối ưu tương đương `useMemo/useCallback/memo` ở những chỗ cần. Mục tiêu: bạn **không phải** tự viết memo nữa.

```
npm install -D babel-plugin-react-compiler
```

```
// vite.config.js
react({ babel: { plugins: ['babel-plugin-react-compiler'] } })
```

Điều kiện để compiler hoạt động: code phải **thuần khiết** theo đúng "Rules of React" — không mutate props/state, không side-effect trong thân component. Nói cách khác: **viết React cho đúng thì được tối ưu miễn phí**. Đây là lý do các quy tắc ở Part 00/03 quan trọng hơn bao giờ hết. Dùng eslint-plugin-react-hooks bản mới để kiểm tra.

10.4 Chia nhỏ bundle: lazy + Suspense

```
const AddJobPage = lazy(() => import('./pages/AddJobPage'));

<Suspense fallback=<Spinner />>
  <AddJobPage />
</Suspense>
```

Trang ít dùng (form thêm/sửa, trang admin) không cần nằm trong bundle đầu tiên. Đo bằng `npm run build` — Vite in ra kích thước từng chunk.

10.5 useTransition — giữ UI không đứng

```
const [isPending, startTransition] = useTransition();

function handleSearch(value) {
  setKeyword(value); // cập nhật gấp: ô input phải phản hồi ngay
  startTransition(() => setQuery(value)); // cập nhật "không gấp": lọc 5000 dòng
}
```

React sẽ ưu tiên cập nhật gấp trước, và có thể **ngắt** cập nhật trong transition nếu có việc gấp hơn tới — nhờ đó ô input không bị nghẽn khi danh sách lớn.

Việc cần làm:

1. Mở React DevTools Profiler, ghi lại một lần gõ tìm kiếm, tìm component render nhiều nhất
2. Tạo 5000 job giả, đo độ trễ khi gõ; áp useMemo cho hàm lọc rồi đo lại
3. Bọc JobListing bằng memo; chứng minh nó chỉ có tác dụng khi hàm truyền xuống được useCallback
4. Dùng lazy + Suspense cho trang add/edit, so sánh kích thước chunk trước/sau
5. Áp useTransition cho ô tìm kiếm với danh sách lớn
6. (Nâng cao) Bật React Compiler và bỏ bớt memo viết tay, đo lại xem còn nhanh không

React 19: Actions, use() & form hooks

Mục tiêu: Nắm những thứ React 19 làm gọn đi so với cách viết cũ — để không học theo tutorial 2022 rồi viết code lỗi thời.

11.1 ref là prop bình thường — không cần useRef

```
// React 19 ✓
function Input({ ref, ...props }) {
  return <input ref={ref} {...props} />;
}

// React 18 và cũ hơn: phải bọc useRef
const Input = useRef((props, ref) => <input ref={ref} {...props} />);
```

Ngoài ra, callback ref giờ có thể trả về hàm cleanup: `ref={(node) => { ...; return () => {...} }}`.

11.2 Context dùng trực tiếp làm Provider

```
<ThemeContext value="dark">{children}</ThemeContext> // React 19
<ThemeContext.Provider value="dark">...</ThemeContext.Provider> // cách cũ, vẫn chạy
```

11.3 Actions: form gọi hàm async trực tiếp

```
// Truyền hàm vào action của <form> — React tự gọi, tự quản lý pending
function AddJobForm() {
  async function createAction(formData) {
    await createJob({
      title: formData.get('title'),
      salary: formData.get('salary'),
    });
  }

  return (
    <form action={createAction}>
      <input name="title" />
      <input name="salary" />
      <SubmitButton />
    </form>
  );
}
```

Form dùng action sẽ **tự reset** sau khi thành công và tự chặn hành vi submit mặc định. Bạn không cần `onSubmit` + `preventDefault` nữa.

11.4 useActionState — state + pending + error trong 1 hook

```
const [state, formAction, isPending] = useActionState(
  async (prevState, formData) => {
    try {
      await createJob({ title: formData.get('title') });
      return { ok: true, error: null };
    } catch (e) {
      return { ok: false, error: e.message }; // trả lỗi về state, không cần useState riêng
    }
  },
  { ok: false, error: null }, // state khởi tạo
);

<form action={formAction}>
  <input name="title" />
  <button disabled={isPending}>{isPending ? 'Đang lưu...' : 'Lưu'}</button>
  {state.error && <p className="text-red-600">{state.error}</p>}
</form>
```

So sánh: cách cũ ở Part 05 cần 3 state (submitting, error, dữ liệu) + try/catch/finally thủ công. Ở đây gói lại trong một hook.

11.5 useFormStatus — nút submit tự biết form đang gửi

```
import { useFormStatus } from 'react-dom'; // chú ý: từ react-dom

function SubmitButton() {
  const { pending } = useFormStatus(); // đọc trạng thái của <form> CHA

  return <button disabled={pending}>{pending ? 'Đang gửi...' : 'Gửi'}</button>;
}
```

Nhờ vậy component nút dùng lại được ở mọi form, không cần truyền prop isPending xuống.

11.6 useOptimistic — cập nhật lạc quan

```
const [optimisticJobs, addOptimistic] = useOptimistic(
  jobs,
  (current, newJob) => [...current, { ...newJob, sending: true }],
);

async function action(formData) {
  const draft = { title: formData.get('title') };

  addOptimistic(draft); // hiện NGAY trên UI
  await createJob(draft); // nếu lỗi → React tự rollback về state thật
}
```

11.7 use() — đọc Promise / Context linh hoạt

```
import { use, Suspense } from 'react';

function JobDetail({ jobPromise }) {
  const job = use(jobPromise); // "mở" promise; đang chờ → Suspense hiện fallback
  return <h1>{job.title}</h1>;
}

<Suspense fallback={<Spinner />}><JobDetail jobPromise={p} /></Suspense>
```

Khác biệt so với hook thường: use() **được** gọi trong if hoặc vòng lặp (các hook khác thì không). Nhưng đừng tạo Promise **trong thân component** (mỗi render tạo mới → loop vô hạn) — promise phải đến từ loader, từ cache, hoặc từ cha.

11.8 Metadata & các tiện ích khác

```
// React 19: đặt <title>, <meta>, <link> ngay trong component — React tự đưa lên <head>
function JobPage({ job }) {
  return (
    <>
      <title>{job.title} | Job Board</title>
      <meta name="description" content={job.description} />
      <h1>{job.title}</h1>
    </>
  );
}
```

API	Tác dụng
<code><Activity mode="hidden"></code>	React 19.2: giữ phần UI ẩn ở trạng thái "tạm dừng" (hủy effect, hoãn cập nhật) để pre-render trước — hữu ích cho tab, modal
<code>useEffectEvent</code>	React 19.2: tách logic sự kiện khỏi dependency của Effect (Part 06)
<code>useId</code>	Sinh id ổn định cho label/input, an toàn khi SSR
Error boundary tốt hơn	Không còn log lỗi trùng lặp; có <code>onCaughtError</code> / <code>onUncaughtError</code> ở <code>createRoot</code>

Việc cần làm:

- Viết lại form thêm job bằng `<form action>` + `useActionState`, so sánh số dòng với bản Part 05
- Tách `SubmitButton` dùng `useFormStatus`, dùng lại ở 2 form khác nhau
- Áp `useOptimistic` cho hành động xóa job: item biến mất ngay, lỗi thì hiện lại
- Đặt `<title>` động cho trang chi tiết, kiểm tra tab trình duyệt đổi tên
- Nếu code cũ còn `forwardRef` hoặc `.Provider` — viết lại theo cú pháp React 19

Kết nối API thật (Laravel / NestJS) & build production

Mục tiêu: Bỏ json-server, nối React vào API bạn tự viết. Đây là phần biến 3 crash course rời rạc thành **một** hệ thống fullstack.

12.1 Bản đồ hệ thống

Browser

React (Vite dev: localhost:5173 · production: file tĩnh trong dist/)

▼ HTTP + JSON

API

Laravel (localhost:8000/api/v1) hoặc NestJS (localhost:3000)

▼

Database

MySQL / PostgreSQL / SQLite

Hai domain khác nhau (5173 ↔ 8000) = **cross-origin** → phải cấu hình CORS ở phía API, nếu không trình duyệt sẽ chặn response.

12.2 CORS — lỗi đầu tiên ai cũng gặp

```
// Thông báo quen thuộc trong console:  
Access to fetch at 'http://localhost:8000/api/v1/jobs' from origin  
'http://localhost:5173' has been blocked by CORS policy
```

Phía	Cách xử lý
Laravel	<code>php artisan config:publish cors</code> → khai <code>allowed_origins => ['http://localhost:5173']</code>
NestJS	<code>app.enableCors({ origin: 'http://localhost:5173' })</code> trong <code>main.ts</code>
React (dev, cách khác)	Dùng proxy của Vite — request cùng origin nên không cần CORS khi dev

```
// vite.config.js — proxy khi dev (mẹo rất tiện)  
server: {  
  proxy: { '/api': { target: 'http://localhost:8000', changeOrigin: true } },  
}  
// Từ React chỉ cần fetch('/api/v1/jobs') — Vite chuyển tiếp sang backend
```

Đừng để `allowed_origins: ['*']` ở production. Và nhớ: CORS là cơ chế của **trình duyệt**, không phải bảo mật server — nó không thay thế cho việc xác thực token.

12.3 Xác thực: lưu token ở đâu

Nơi lưu	Ưu	Nhược
localStorage	Đơn giản, sống qua refresh	Độc được bằng JS → bị đánh cắp nếu có lỗ hổng XSS
Memory (state/context)	An toàn nhất với XSS	Mất khi refresh trang
Cookie HttpOnly	JS không đọc được → chống XSS	Cần backend set cookie + lo CSRF; phải cùng site

Với crash course: dùng localStorage + AuthContext (Part 08) cho đơn giản, nhưng **phải biết** đây là đánh đổi. Dự án thật nên dùng cookie HttpOnly do backend set (Sanctum SPA mode của Laravel làm đúng việc này).

```
// Gắn token vào mọi request – sửa hàm request() ở Part 07
const token = localStorage.getItem('token');

const res = await fetch(url, {
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json', // BẮT BUỘC với Laravel để nhận lỗi JSON
    ...(token && { Authorization: `Bearer ${token}` }),
  },
});

if (res.status === 401) { // token hết hạn
  localStorage.removeItem('token');
  window.location.href = '/login';
}
```

12.4 Map lỗi API sang UI

Status	Ý nghĩa (từ Part 10 của tài liệu Laravel)	UI nên làm gì
422	Validation sai, có errors theo từng field	Hiện lỗi ngay dưới đúng field đó
401	Chưa/không còn đăng nhập	Xoá token, chuyển tới trang login
403	Không có quyền	Thông báo "bạn không có quyền", không chuyển login
404	Không tồn tại	Trang "không tìm thấy"
429	Quá nhiều request	Báo thử lại sau, đọc header Retry-After
500	Lỗi server	Thông báo chung + nút thử lại. Không hiện chi tiết lỗi cho user

```
// Nhận lỗi 422 của Laravel và đổ vào form
if (res.status === 422) {
  const { message, errors } = await res.json();
  setFieldErrors(errors); // { title: ['...'], salary: ['...'] }
  return;
}
```

12.5 Build & deploy

```
npm run build      # → dist/ : index.html + assets có hash
npm run preview    # chạy thử bản build ở local
```

- **SPA fallback:** server phải trả `index.html` cho **mọi** đường dẫn, nếu không F5 tại `/jobs/5` sẽ ra 404. Nginx: `try_files $uri /index.html;`
- **Biến môi trường được nhúng lúc build** — đổi `VITE_API_URL` thì phải build lại.
- Đặt React và API sau **cùng một domain** (vd `app.example.com` + `app.example.com/api`) để khỏi phải lo CORS và dùng được cookie `HttpOnly`.

Việc cần làm:

1. Chạy song song: API Laravel (hoặc NestJS) ở cổng 8000 và React ở 5173
2. Sửa `jobService.js` trở sang API thật qua `VITE_API_URL`
3. Gặp lỗi CORS → sửa ở phía API; sau đó thử lại bằng cách dùng proxy của Vite, hiểu khác biệt hai cách
4. Làm trang login gọi API thật, lưu token, gắn Bearer vào mọi request tiếp theo
5. Xử lý 401 (tự logout), 403, 422 (đổi lỗi vào field), 500 (thông báo chung)
6. Bảo vệ route: chưa đăng nhập thì `/add-job` tự chuyển sang `/login`
7. Build production, chạy preview, F5 tại URL con để kiểm tra SPA fallback

Testing với Vitest + React Testing Library

Mục tiêu: Test component theo cách **người dùng thật** tương tác, chứ không test chi tiết cài đặt bên trong.

13.1 Cài đặt

```
npm i -D vitest @testing-library/react @testing-library/user-event \
  @testing-library/jest-dom jsdom
```

```
// vite.config.js
test: { environment: 'jsdom', globals: true, setupFiles: './src/setupTests.js' }

// src/setupTests.js
import '@testing-library/jest-dom';
```

13.2 Triết lý: test như người dùng

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';

test('hiện tiêu đề job', () => {
  render(<JobListing job={{ id: 1, title: 'React Dev' }} />);

  expect(screen.getByText('React Dev')).toBeInTheDocument();
});

test('gõ tìm kiếm thì lọc danh sách', async () => {
  const user = userEvent.setup();
  render(<JobsPage />);

  await user.type(screen.getByRole('textbox', { name: /tìm/i }), 'react');

  expect(await screen.findByText('React Dev')).toBeInTheDocument();
  expect(screen.queryByText('PHP Dev')).not.toBeInTheDocument();
});
```

Query	Khi nào dùng
<code>getByRole</code>	Ưu tiên số 1 — giống cách người dùng & screen reader tìm phần tử
<code>getByLabelText</code>	Input trong form (đồng thời kiểm tra accessibility)
<code>getByText</code>	Nội dung hiển thị
<code>findBy*</code>	Phần tử xuất hiện sau khi async xong (tự chờ)
<code>queryBy*</code>	Khi cần khẳng định không tồn tại (không throw)
<code>getByTestId</code>	Phương án cuối — dùng nhiều nghĩa là markup chưa accessible

Đừng test cái này: state nội bộ, tên hàm, số lần render. Hãy test **hành vi**: "gõ vào ô này thì thấy kết quả kia", "bấm xóa thì item biến mất". Nhờ vậy bạn refactor bên trong mà test vẫn xanh.

13.3 Mock API

```
// Cách nhanh: mock fetch
global.fetch = vi.fn(() =>
  Promise.resolve({ ok: true, json: () => Promise.resolve([ { id: 1, title: 'React Dev' } ] ) }));

// Cách chuẩn hơn cho dự án thật: MSW (Mock Service Worker) – giả lập ở tầng network
```

Việc cần làm (tuỳ chọn):

1. Cấu hình Vitest + jsdom, chạy được `npm run test`
2. Test `JobListing` render đúng title/type/salary
3. Test form thêm job: bỏ trống title → hiện lỗi validate
4. Test danh sách với fetch được mock: loading → có dữ liệu
5. Test nhánh lỗi: fetch reject → hiện thông báo lỗi

— Hết tài liệu — Quay lại chat khi bạn xong mỗi Part để được review & giải thích tiếp phần sau —